Long Short-Term Memory Networks

From neural networks to LSTMs — with an application to Jamaica's Real GDP Growth

Jose Luis Saboin, PhD
Inter-American Development Bank

Real GDP Nowcasting Tool for BoJ

May 5, 2026

"How neural networks remember the past to predict the future."

Part I — Neural Networks

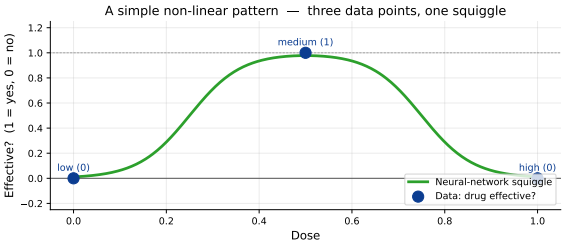
What is a neural network?

A neural network is a **flexible function approximator** — a "squiggle-fitting machine".

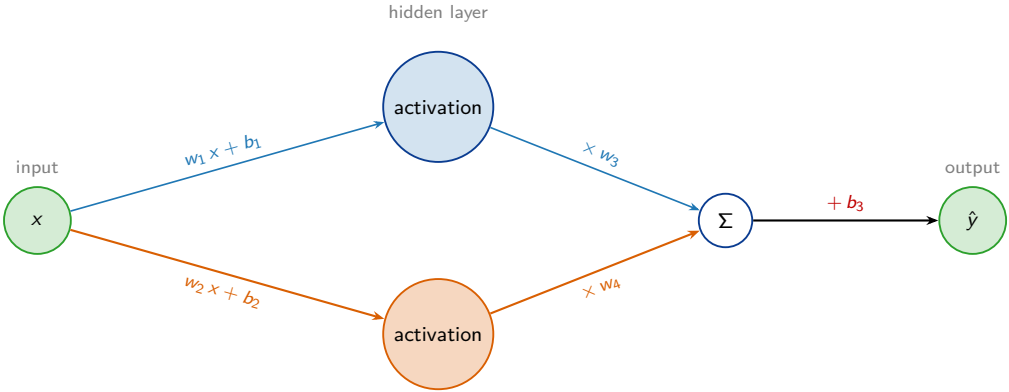
Toy example. Three drug-dose observations:

- low (0.0) $\rightarrow$ not effective (0)
- medium (0.5) $\rightarrow$ effective (1)
- high (1.0) $\rightarrow$ not effective (0)

A straight line cannot fit all three.
A neural network can — by combining a few *activation functions* into a non-linear curve.

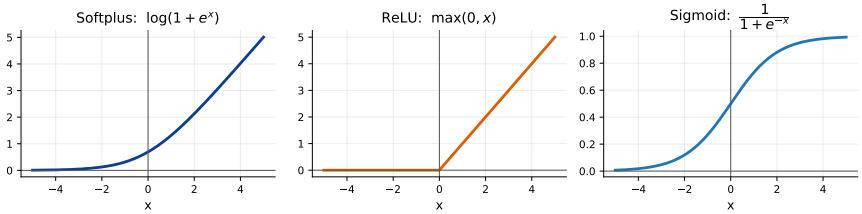


Anatomy of a neural network



- **Nodes** = neurons
- **Edges** = weighted connections (w)
- Each hidden node adds a bias (b) and applies an **activation function**
- Stacking layers $\Rightarrow$ deep networks
- Total parameters here: $w_1, \ldots, w_4$ and b_1, b_2, b_3 = **7**

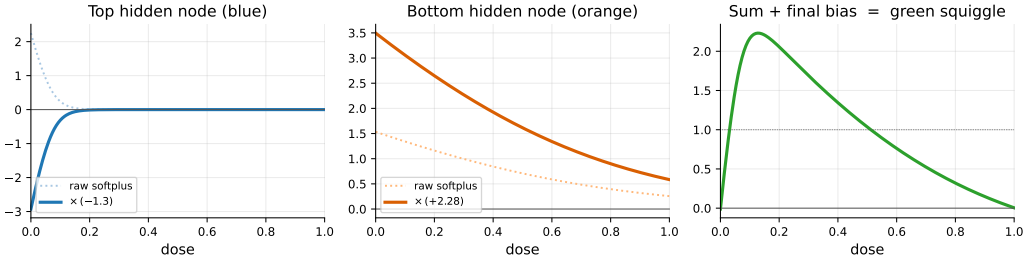
Activation function menu



- **Softplus** — smooth "soft ReLU"; used in this deck's worked examples.
- **ReLU** — cheap and robust; the modern default for deep networks.
- **Sigmoid** — squashes to $(0,1)$; used as a *gating* signal in LSTMs.

How a neural network builds a curve

Each hidden node takes a copy of the activation function and **slices, flips, and stretches it**. Adding the resulting curves produces the final shape.



- Top node: $-1.3 \cdot \log\left(1 + e^{-34.4\,x + 2.14}\right)$
- Bottom node: $+2.28 \cdot \log\left(1 + e^{-2.52\,x + 1.29}\right)$
- Sum + final bias $b_3 = -0.58 \;\Rightarrow\;$ the green squiggle.

How do we choose the parameters?

The network has **seven unknowns** $(w_1, \ldots, w_4, b_1, b_2, b_3)$.

Strategy.

1. Start with random values for the parameters.
2. Define a **loss function** that measures how badly the network fits the data (e.g. Sum of Squared Residuals: $\mathrm{SSR} = \sum_i (y_i - \hat{y}_i)^2$).
3. Compute how the loss changes when each parameter changes.
4. Take a step in the direction that reduces the loss.
5. Repeat.

To do this we need two ingredients:

- The **chain rule** — to compute derivatives through layers (Part II).
- **Gradient descent** — to take iterative downhill steps (Part III).

Combine both $\Rightarrow$ **back-propagation** (Part IV).

Part II — The Chain Rule

Chain rule — a linear example

Suppose three quantities are linked:

$$\text{weight} \;\rightarrow\; \text{height} \;\rightarrow\; \text{shoe}$$

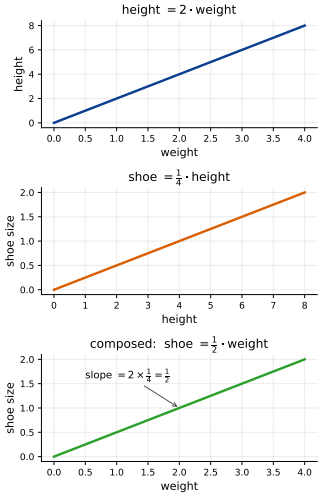
with

$$\text{height} = 2 \cdot \text{weight}, \qquad \text{shoe} = \frac{1}{4} \cdot \text{height}.$$

where:
weight — input; height — intermediate; shoe — output.
2 and $\frac{1}{4}$ are slopes (constant derivatives).

Then a unit change in weight produces:

$$\frac{\text{dshoe}}{\text{dweight}} = \underbrace{\frac{\text{dshoe}}{\text{dheight}}}_{1/4} \cdot \underbrace{\frac{\text{dheight}}{\text{dweight}}}_{2} = \frac{1}{2}.$$



Chain rule — a non-linear example

Now let

$$\mathrm{hunger}(t) = t^2 + \tfrac{1}{2}, \quad \mathrm{craves}(h) = \sqrt{h}.$$

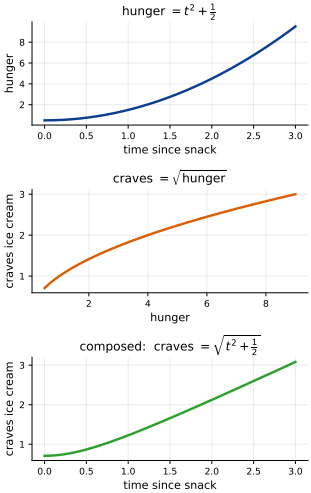
Composed: $\mathrm{craves}(t) = \sqrt{t^2 + \frac{1}{2}}$. **where:**

t — time since last snack; h — hunger level.

Both hunger' and craves' depend on the current value.

Chain rule:

$$\frac{\mathrm{dcraves}}{\mathrm{d}t} = \frac{\mathrm{dcraves}}{\mathrm{d}h} \cdot \frac{\mathrm{d}h}{\mathrm{d}t}$$
$$= \frac{1}{2\sqrt{h}} \cdot 2t = \frac{t}{\sqrt{t^2 + 1/2}}.$$



The chain rule works for any composed function — we will use it on every parameter inside a neural network.

Chain rule on a loss function

Setup. Fit a line $\hat{y} = b + m\,x$ to one observation (x, y). Loss = squared residual:

$$L(b) = (y - \hat{y})^2 = (y - b - m\,x)^2.$$

Define the residual $r = y - \hat{y}$. Then $L = r^2$ and r depends on b.

Chain rule:

$$\frac{\partial L}{\partial b} = \underbrace{\frac{\partial L}{\partial r}}_{2r} \cdot \underbrace{\frac{\partial r}{\partial b}}_{-1} = -2\,(y - \hat{y}).$$

where:
L — loss; r — residual; b — intercept (what we're optimising).

The factor $-2(y - \hat{y})$ is the **error signal** — it will reappear everywhere in back-propagation.

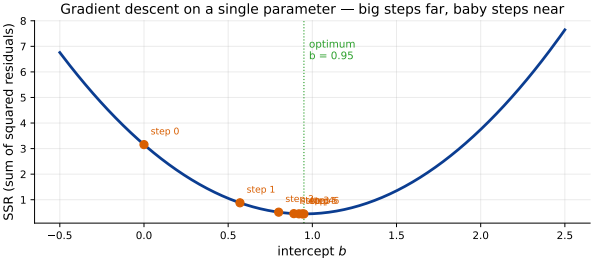
This derivative tells us **which direction to nudge b to reduce the loss**.

Part III — Gradient Descent

Gradient descent — one parameter

Algorithm.

1. Pick a random initial value b_0.
2. Compute slope $g = \frac{\partial L}{\partial b}$ at the current b.
3. Step size $= g \cdot \eta$ (η = learning rate).
4. $b_{\text{new}} = b - g \cdot \eta$.
5. Repeat until step size < tolerance or max-iterations reached.



Why the step shrinks naturally: as b approaches the optimum, the slope g goes to zero, so the step size shrinks — **big steps far away, baby steps close**.

Gradient descent — many parameters

With K parameters $\theta = (\theta_1, \ldots, \theta_K)$:

$$\nabla L = \left(\frac{\partial L}{\partial \theta_1}, \ldots, \frac{\partial L}{\partial \theta_K} \right)$$

is the **gradient**. The update is

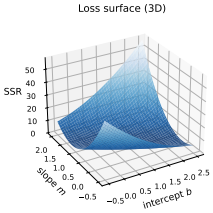
$$\theta_{\text{new}} = \theta - \eta \cdot \nabla L.$$

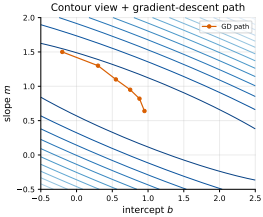
where:

θ — parameter vector; ∇L — vector of partial derivatives; η — learning rate (controls step size).

Each partial derivative measures how much the loss changes when *only that one parameter* moves; all the others are held fixed.

Two parameters (b, m) define a loss *surface*. Gradient descent rolls the orange ball downhill, one step at a time.





Part IV — Back-propagation

Back-propagation: optimising one parameter

Setup. Assume all parameters are already optimal except the last bias b_3. The full prediction unrolls all 7 parameters:

$$\hat{y}_i \;=\; w_3 \cdot \log\!\left(1 + e^{w_1 x_i + b_1}\right) \;+\; w_4 \cdot \log\!\left(1 + e^{w_2 x_i + b_2}\right) \;+\; b_3.$$

Loss is $\mathrm{SSR} = \sum_i (y_i - \hat{y}_i)^2$. The chain rule gives

$$\frac{\partial \mathrm{SSR}}{\partial b_3} \;=\; \sum_i \underbrace{\frac{\partial (y_i - \hat{y}_i)^2}{\partial \hat{y}_i}}_{-2(y_i - \hat{y}_i)} \cdot \underbrace{\frac{\partial \hat{y}_i}{\partial b_3}}_{1} \;=\; \sum_i -2(y_i - \hat{y}_i).$$

where:
$\hat{y}_i$ — network's prediction for input i; y_i — observed value.

Holding $w_1, b_1, w_2, b_2, w_3, w_4$ fixed, only b_3 is variable here, so $\partial \hat{y}_i / \partial b_3 = 1$ and the rest of the formula contributes zero.

Plug-and-chug. Initialise $b_3 = 0$, compute slope, take a step, repeat $\Rightarrow$ converges to optimal $b_3 \approx 2.61$ in a few steps.

Back-propagation: many parameters at once

Now extend to w_3, w_4, b_3 (the last three parameters). Chain rule for each:

$$\frac{\partial \mathrm{SSR}}{\partial w_3} = \sum_i \underbrace{\frac{\partial (y_i - \hat{y}_i)^2}{\partial \hat{y}_i}}_{-2(y_i - \hat{y}_i)} \cdot \underbrace{\frac{\partial \hat{y}_i}{\partial w_3}}_{y_{1,i}} = \sum_i -2(y_i - \hat{y}_i) \cdot y_{1,i}.$$

$$\frac{\partial \mathrm{SSR}}{\partial w_4} = \sum_i -2(y_i - \hat{y}_i) \cdot y_{2,i}, \qquad \frac{\partial \mathrm{SSR}}{\partial b_3} = \sum_i -2(y_i - \hat{y}_i).$$

where:

$y_{1,i} = \log(1 + e^{w_1 x_i + b_1})$ — top hidden-node output (carries w_1, b_1); $y_{2,i} = \log(1 + e^{w_2 x_i + b_2})$ — bottom (carries w_2, b_2); rest as before.

Key observation. The factor $-2(y_i - \hat{y}_i)$ — the **error signal** — appears in *every* partial derivative. Each parameter's gradient = (shared error signal) $\times$ (its own local derivative).

This is what makes back-propagation efficient: **shared work is computed once and reused**.

Back-propagation: the full chain rule

For an upstream weight w_1, the chain has *four* factors:

$$\frac{\partial \mathrm{SSR}}{\partial w_1} = \underbrace{\frac{\partial \mathrm{SSR}}{\partial \hat{y}}}_{-2(y-\hat{y})} \cdot \underbrace{\frac{\partial \hat{y}}{\partial y_1}}_{w_3} \cdot \underbrace{\frac{\partial y_1}{\partial x_1}}_{\sigma(x_1)} \cdot \underbrace{\frac{\partial x_1}{\partial w_1}}_{\text{input}} = -2(y-\hat{y}) \cdot w_3 \cdot \sigma(x_1) \cdot \mathrm{input}.$$

where:
$x_1 = w_1 \cdot \mathrm{input} + b_1$; $y_1 = \log(1 + e^{x_1})$; $\hat{y} = w_3 y_1 + w_4 y_2 + b_3$.

The factor $\sigma(x_1)$ in the chain product equals $\partial y_1 / \partial x_1 = e^{x_1}/(1 + e^{x_1})$ — the sigmoid emerges from differentiating the softplus.

Recursive structure. Going further upstream just adds another factor to the chain. Modern networks (millions of parameters) compute *all* of them in two passes:

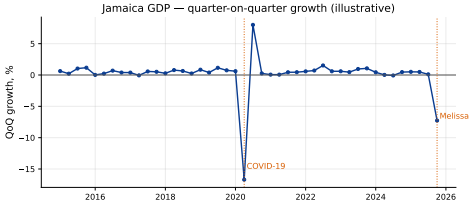
- **Forward pass** — compute every hidden-node value (the x's and y's) and the prediction $\hat{y}$.
- **Backward pass** — compute the partial derivative of SSR with respect to *each* parameter, reusing intermediate factors of the chain.

Full algorithm: forward pass $\rightarrow$ compute loss $\rightarrow$ backward pass $\rightarrow$ update parameters $\rightarrow$ repeat

Part V — Recurrent Networks

Why memory matters in a forecasting model

- GDP at quarter t depends on the **history** through $t-1$.
- Observations are *not* independent — yesterday's tourism shock matters today.
- Feed-forward networks cannot "remember" across observations.
- We need a **recurrent** model: one that carries state forward in time.



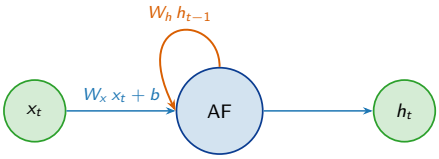
Sequential model in one line:

$$\hat{y}_t \;=\; f\big(x_t,\, \mathrm{state}_{t-1}\big).$$

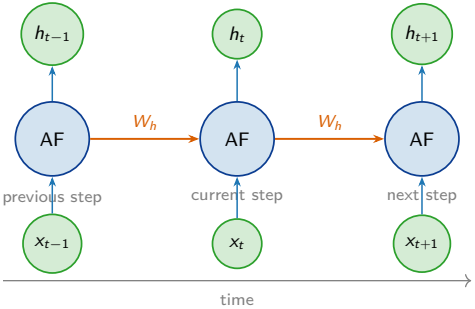
where:

x_t — input at time t; state_{t-1} — summary of all past observations.

Anatomy of a recurrent neural network



(rolled: feedback loop)



- **Input x_t, hidden state h_t, recurrence** $h_{t-1} \to h_t$
- Same parameters (W_x, W_h, b) *shared at every time step*
- Handles **any sequence length** with the same model
- One unit, unrolled T times = a T-step network

where: W_h — weight on the previous hidden state (the recurrent weight); AF — activation function.

Vanilla RNNs and the gradient problem

An RNN has a **feedback loop**: the hidden state at t becomes part of the input at $t{+}1$.

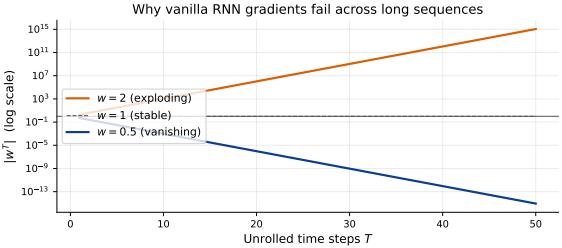
$$h_t = \mathrm{AF}\big(W_x\, x_t + W_h\, h_{t-1} + b\big).$$

h_t: hidden state at t; x_t: input at t; W_x, W_h, b: learned; AF: activation function.

Unrolled across T steps, the gradient through time scales as

$$\frac{\partial h_T}{\partial h_0} \;\propto\; {W_h}^T.$$

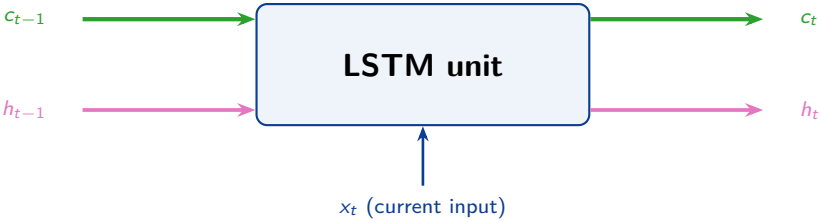
- $|W_h| > 1$: gradient **explodes**.
- $|W_h| < 1$: gradient **vanishes**.



Part VI — Long Short-Term Memory

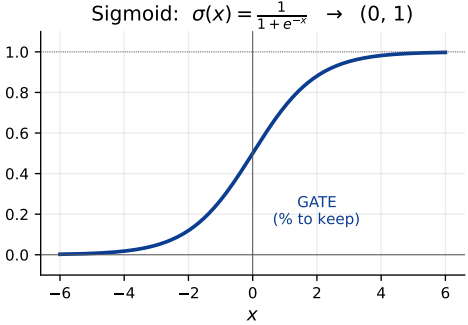
The LSTM idea: two memory paths

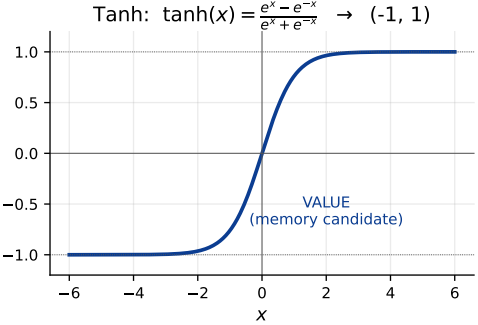
Hochreiter & Schmidhuber (1997). Each unit carries *two* memory streams forward — one long-term, one short-term — and updates them differently.



- **Cell state** (long-term): updated by additions and gate-controlled multiplications only — **no weight matrix multiplies it through time**, so the gradient survives.
- **Hidden state** (short-term): the unit's output, weight-modulated like in a vanilla RNN.

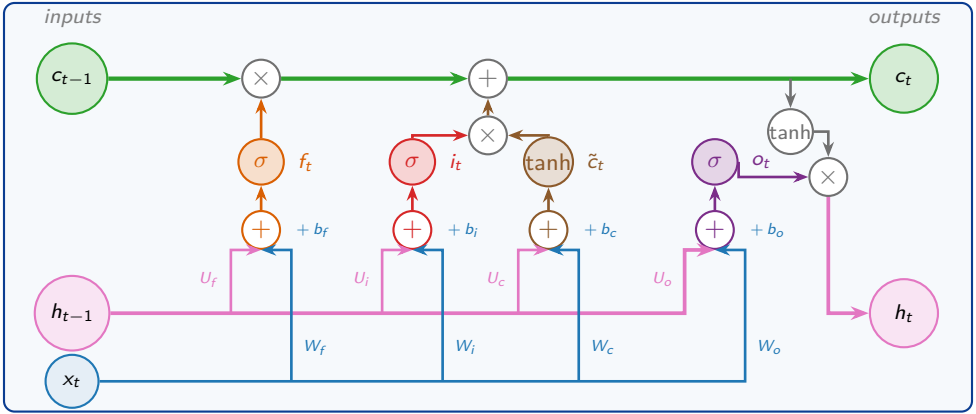
Building blocks: σ and tanh





Function	Range	Role inside the LSTM
Sigmoid $\sigma(x)$	$(0, 1)$	**Gate** — "what fraction to keep / write"
$\tanh(x)$	$(-1, 1)$	**Value** — "candidate memory content"

The LSTM unit: three gates, two memory paths



$\boldsymbol{h_t}$ is the LSTM's output at time t — the value passed forward as h_{t-1} to the next step.

Every σ gate outputs a value in $(0, 1)$ — a **percentage** of how much to let through:

Forget (f_t, *% of* c_{t-1} *kept*) | **Input** (i_t, *% of new info written*) | **Candidate** ($\tilde{c}_t$, *the new info itself*) | **Output** (o_t, *% of* $\tanh(c_t)$ *revealed as* h_t).

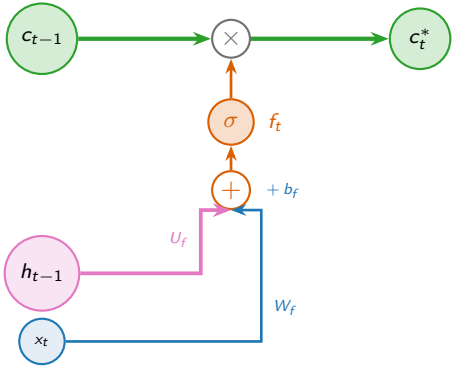
Stage 1 — Forget gate: *what to keep from yesterday*

$$f_t = \sigma\big(U_f\, h_{t-1} + W_f\, x_t + b_f\big),$$

$$c_t^* = f_t \odot c_{t-1}.$$

where:

- f_t — forget-gate output, in $(0, 1)$
- U_f, W_f, b_f — learned weights for h_{t-1}, x_t, and bias
- $\odot$ — element-wise product
- c_{t-1} — prior long-term memory
- c_t^* — partially-forgotten memory



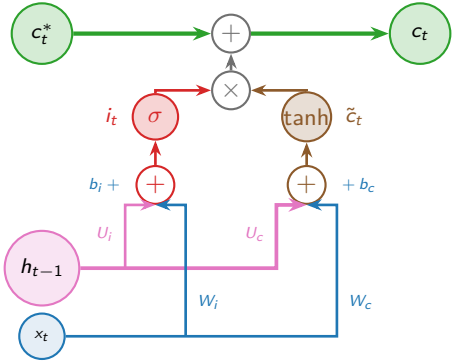
Intuition: f_t is a per-element **valve**.

- $f_t \approx 1 \rightarrow$ keep the long-term memory.
- $f_t \approx 0 \rightarrow$ forget it.

Worked example. $h_{t-1} = 1,\ x_t = 1,\ U_f = 2.7,\ W_f = 1.63,\ b_f = 1.62 \Rightarrow$
$f_t = \sigma(2.7 + 1.63 + 1.62) = \sigma(5.95) = 0.997 \Rightarrow c_t^* = 0.997 \times 2 = 1.99.$

Stage 2 — Input gate: *what new information to add*

$$
\begin{aligned}
i_t &= \sigma\big(U_i\, h_{t-1} + W_i\, x_t + b_i\big) \\
\tilde{c}_t &= \tanh\big(U_c\, h_{t-1} + W_c\, x_t + b_c\big) \\
c_t &= c_t^* + i_t \odot \tilde{c}_t .
\end{aligned}
$$



where:

- i_t — input-gate output, in $(0, 1)$
- $\tilde{c}_t$ — **candidate** new memory value, in $(-1, 1)$
- U_i, U_c, W_i, W_c, b_i, b_c — learned weights and biases
- c_t — updated long-term memory

Two parts: $\tilde{c}_t \in (-1, 1)$ is the **candidate value**;
$i_t \in (0, 1)$ is the **write-fraction** gate.

Worked example. $\tilde{c}_t = \tanh(2.03) = 0.97$, $i_t = \sigma(4.27) = 1.00 \Rightarrow$
$c_t = c_t^* + i_t \odot \tilde{c}_t = 1.99 + 1.0 \times 0.97 = 2.96.$

Stage 3 — Output gate: *what to emit as today's signal*

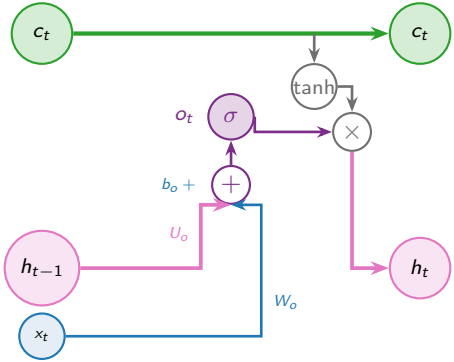
$$o_t = \sigma\big(U_o\, h_{t-1} + W_o\, x_t + b_o\big),$$

$$h_t = o_t \odot \tanh(c_t).$$

where:

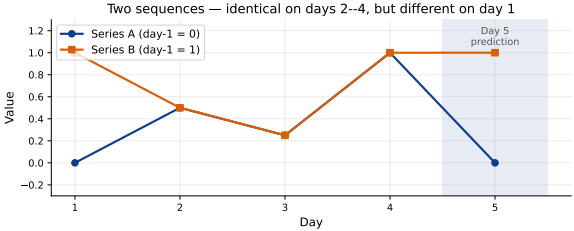
- o_t — output-gate value, in $(0, 1)$
- U_o, W_o, b_o — learned weights and bias
- h_t — new short-term memory *and* unit output at time t

$o_t \in (0, 1)$ decides what **fraction of the squashed cell state** is exposed.



Worked example. $\tanh(c_t) = \tanh(2.96) = 0.99$, $o_t = 0.99 \Rightarrow$
$h_t = o_t \odot \tanh(c_t) = 0.99 \times 0.99 = 0.98.$

Two scenarios — same LSTM, different day-5 forecasts



- Series A: input [0, 0.5, 0.25, 1] predicts day 5 = **0**.
- Series B: input [1, 0.5, 0.25, 1] predicts day 5 = **1**.
- Days 2–4 are *identical* between A and B.

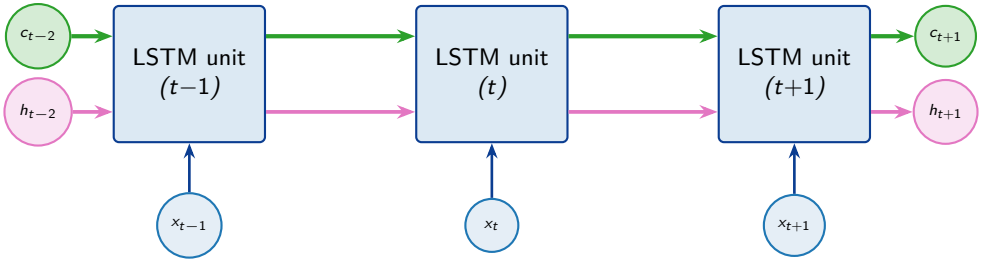
What this shows.

The cell state c_t encodes the day-1 difference and **carries it intact through 4 unrolled steps**.

The same weights (W_f, W_i, W_c, W_o) produce *different* outputs because the *long-term memory* differs.

This is the **long-short distinction** doing work.

Unrolling: same weights, T time steps



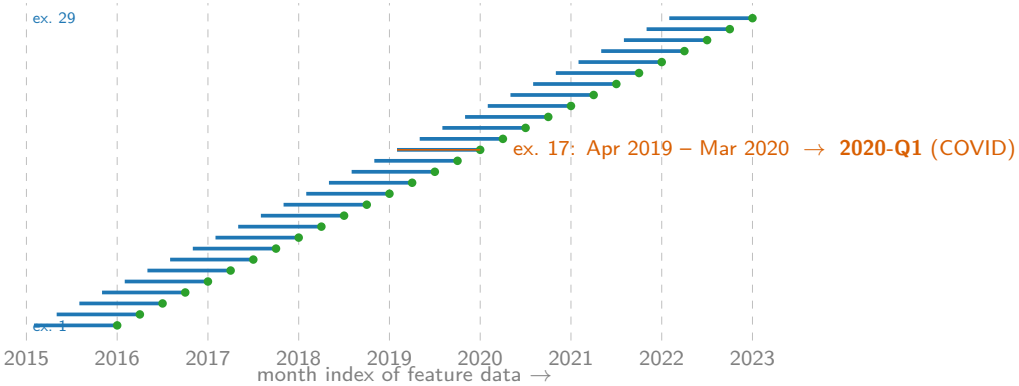
- Same matrices (W_f, W_i, W_c, W_o) **at every time step**.
- Network learns "what to do at every step", not "what each step does".
- Parameter sharing $\Rightarrow$ the same model handles **any sequence length**.
  - In the BoJ pipeline: `n_timesteps = 12` $\Rightarrow$ 12 months of feature history per nowcast.

Part VII — The `nowcast_lstm` library

LSTM library knobs — `nowcast_lstm` hyperparameters

Parameter	Pipeline value	Intuition
`n_timesteps`	12	How many steps back the LSTM looks. Longer = more context but more compute and harder optimisation.
`n_models`	50	**Bagging**: train 50 LSTMs with different random initialisations and average. Variance reduction ⇒ smoother forecasts.
`train_episodes` (epochs)	100	**Number of full passes over the training data**. Too few → underfit; too many → overfit and drift.
`batch_size`	50	Number of training sequences per gradient step. Bigger = smoother gradient, more memory.
`decay`	0.98	Per-epoch **learning-rate decay**. The optimiser slows down as it approaches a minimum.
`n_hidden`	10	Width of the hidden & cell state. Sets the **model capacity** per layer.
`n_layers`	1	Stacked LSTM depth. >1 learns hierarchical temporal features (we use 1 for our small data).
`dropout`	0.0	Random unit dropout for regularisation. We rely on bagging instead.
`criterion`	MSE	Loss function: average squared $\hat{y} - y$ error.
`optimizer` (η)	Adam (η=0.01)	Gradient-descent variant; η is the initial learning rate.

The 29 sliding 12-month training windows



Each **blue bar** is a 12-month feature window; the **green dot** at its right end is the quarterly GDP it predicts. Consecutive windows shift by 3 months and share 9 months of features — the windows are highly overlapping, which is why *bagging* matters.

How the pipeline trains: nested loops over epochs and seeds

Outer loop ($\times 50$ random seeds):

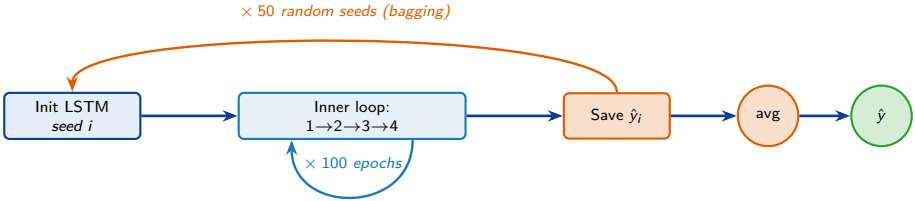
1. Init LSTM with random weights (seed i).
2. Run the inner loop (right) for 100 epochs.
3. Save this seed's prediction $\hat{y}_i$.

After the 50 outer iterations:

$$\hat{y} = \frac{1}{50} \sum_{i=1}^{50} \hat{y}_i.$$

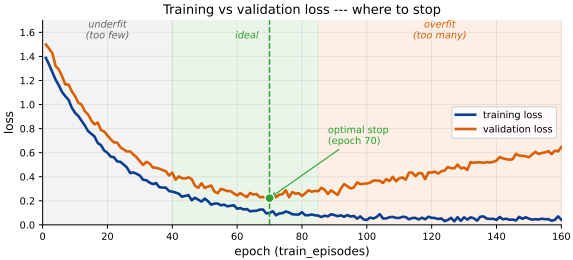
Inner loop ($\times 100$ epochs):

1. Forward-pass all 30 examples through the LSTM $\rightarrow$ 30 predictions.
2. Average the 30 squared errors $\rightarrow$ scalar MSE loss.
3. Back-prop $\rightarrow$ gradient of loss w.r.t. every weight.
4. Update weights: $w \leftarrow w - \eta \cdot \text{grad}$.



Why `train_episodes` matters: a quick picture

- One "episode" (epoch) = the optimiser sees **the entire training set once**.
- Loss falls quickly at first.
- Then it plateaus as the model nears the best fit.
- **Too few epochs** → under-trained model.
- **Too many epochs** → over-fits to the training noise; out-of-sample drifts.
- Bagging across 50 random inits hides much of this — but tuning epochs still matters.



Summary

- Neural networks are flexible function approximators built from activation functions.
- The **chain rule** computes derivatives through composed functions.
- **Gradient descent** uses those derivatives to walk downhill on the loss.
- Together they form **back-propagation**, which optimises every parameter in the network.
- Vanilla RNNs cannot remember across long sequences (gradient explodes / vanishes).
- LSTMs add a **cell state** that flows with only $\times$ and $+$ — gradients survive.
- Three sigmoid **gates** (forget, input, output) and a tanh **candidate value** per unit.
- In our pipeline: 12-step lookback, 50-model bag, 100 epochs $\rightarrow$ stable monthly-to-quarterly nowcasts.

References: Hochreiter & Schmidhuber (1997); Gers, Schmidhuber & Cummins (2000); Bengio, Simard & Frasconi (1994); Olah, "Understanding LSTM Networks" (2015); Starmer, StatQuest series.